



Trabajo Veterinaria

Rojas Camargo Betsy Juliana

Bueno Ceballes Cesar David

Doncel Rodríguez David Alejandro

Bonilla Alisson Paola

Ingeniería de sistemas

Sexto semestre

Docente

Julián Alberto Prado Sánchez

Bogotá D.C. – 2026

Resumen

En este proyecto desarrollamos una Veterinaria, una aplicación de escritorio sencilla para solucionar un problema cotidiano: el desorden que tienen los dueños de mascotas con los carnets de vacunación y la solicitud de citas en veterinarias o guarderías. El programa se construyó en Python aplicando Programación Orientada a Objetos (POO), arquitectura MVC (Modelo-Vista-Controlador), una base de datos local en SQL para no perder la información al cerrar, y una interfaz visual hecha llamativamente volviendo la app fácil de usar. Usamos un ciclo de vida en cascada para organizar el trabajo desde la toma de requerimientos hasta las pruebas. El resultado es un sistema funcional que evita que a la gente se le pasen las fechas de las vacunas (usando un semáforo de colores) y controla los cupos de la guardería para que no se crucen las reservas.

Palabras clave: Python, POO, SQL, Ingeniería de Software.

Abstract

For this project, we developed "Veterinaria," a simple desktop application designed to solve a common problem: the disorganization pet owners face regarding vaccination records and the process of booking appointments at veterinary clinics or pet daycares. The program was built using Python and Object-Oriented Programming (OOP) principles, employing the MVC (Model-View-Controller) architecture. It utilizes a local SQL database to ensure data persistence and features an eye-catching, user-friendly visual interface. We followed a waterfall development lifecycle to organize the workflow, spanning from requirements gathering to testing. The result is a functional system that helps prevent missed vaccination dates (using a color-coded status indicator) and manages daycare capacity to avoid booking conflicts.

Keywords: Python, OOP, SQL, Software Engineering.

Introducción

Hoy en día muchísima gente tiene perros o gatos en la casa y los cuida como un miembro más de la familia. El problema es que casi todo el control médico (vacunas, desparasitantes, peso) se sigue llevando en cuadernos o carnets de papel que siempre se terminan perdiendo o rompiendo. Además, cuando la gente quiere llevar a la mascota a la guardería o al baño, todo se maneja por chats de WhatsApp, lo que casi siempre termina en desorden de horarios y cupos llenos.

Pensando en eso, creamos La veterinaria. La idea del proyecto es aplicar lo que hemos visto en la carrera (diseño de clases, bases de datos relacionales e interfaces de usuario) para hacer una herramienta que centralice la información de la mascota, le avise al usuario de forma clara cuándo le toca la próxima vacuna y le permita pedir citas sin complicaciones.

Planteamiento del problema

Detectamos tres fallas principales en cómo la gente cuida a sus mascotas hoy en día:

- Los carnets de papel no sirven: Se pierden, se mojan o se olvidan en la casa justo el día que se necesitan.
- La gente se descuida con los refuerzos: Como no hay recordatorios, a los dueños se les olvida la fecha exacta de la siguiente vacuna o desparasitación.
- Desorden en las reservas: Las veterinarias y guarderías pequeñas sufren porque aceptan clientes sin controlar bien el aforo del día.

Pregunta del Proyecto

¿Cómo podemos programar un sistema fácil de usar en Python que le organice la salud a la mascota, le recuerde sus vacunas y controle los cupos de las citas?

Objetivos

Objetivo General

Programar una aplicación de escritorio en Python que permita registrar el historial médico de las mascotas, alertar sobre las fechas de vacunación y agendar citas de guardería y baño, aplicando buenas prácticas de ingeniería de software.

Objetivos Específicos

Levantar y documentar los requerimientos funcionales y no funcionales del sistema.

Diseñar el diagrama de clases (POO) y los casos de uso para estructurar el código antes de programar.

Crear la interfaz gráfica (GUI) e integrarla con la base de datos SQLite3 para guardar la información.

Probar que el sistema funcione bien, verificando que los cupos no se crucen y que las alertas de vacunas calculen bien los días.

Estructura del Proyecto

La organización técnica del sistema se estructuró mediante una arquitectura modular en español basada en el patrón Modelo-Vista-Controlador:

- **configuración:** Contiene el módulo `base_datos.py`, encargado de gestionar la conexión y creación de tablas en SQLite3.
- **modelos:** Agrupa las clases principales del sistema (`usuario.py`, `mascota.py`, `vacuna.py`, `reserva.py`), las cuales representan las entidades del diagrama de clases y manejan los datos del negocio.
- **vistas:** Contiene los archivos de la interfaz gráfica de usuario (`login_vista.py`, `mascota_vista.py`, `citas_vista.py`) con los que interactúa el cliente.
- **controladores:** Agrupa la lógica intermedia (`usuario_controlador.py`, `mascota_controlador.py`, `reserva_controlador.py`) que conecta las peticiones de la interfaz con la base de datos.
- **principal.py:** Es el archivo principal de ejecución que inicia la aplicación de escritorio.

Justificación de la organización: Esta separación por capas permite aislar la interfaz visual de la base de datos local. De esta manera, el código queda organizado de forma clara para el trabajo colaborativo en GitHub y facilita el mantenimiento del software a medida que se implementen las funcionalidades.

Lógica del proyecto:

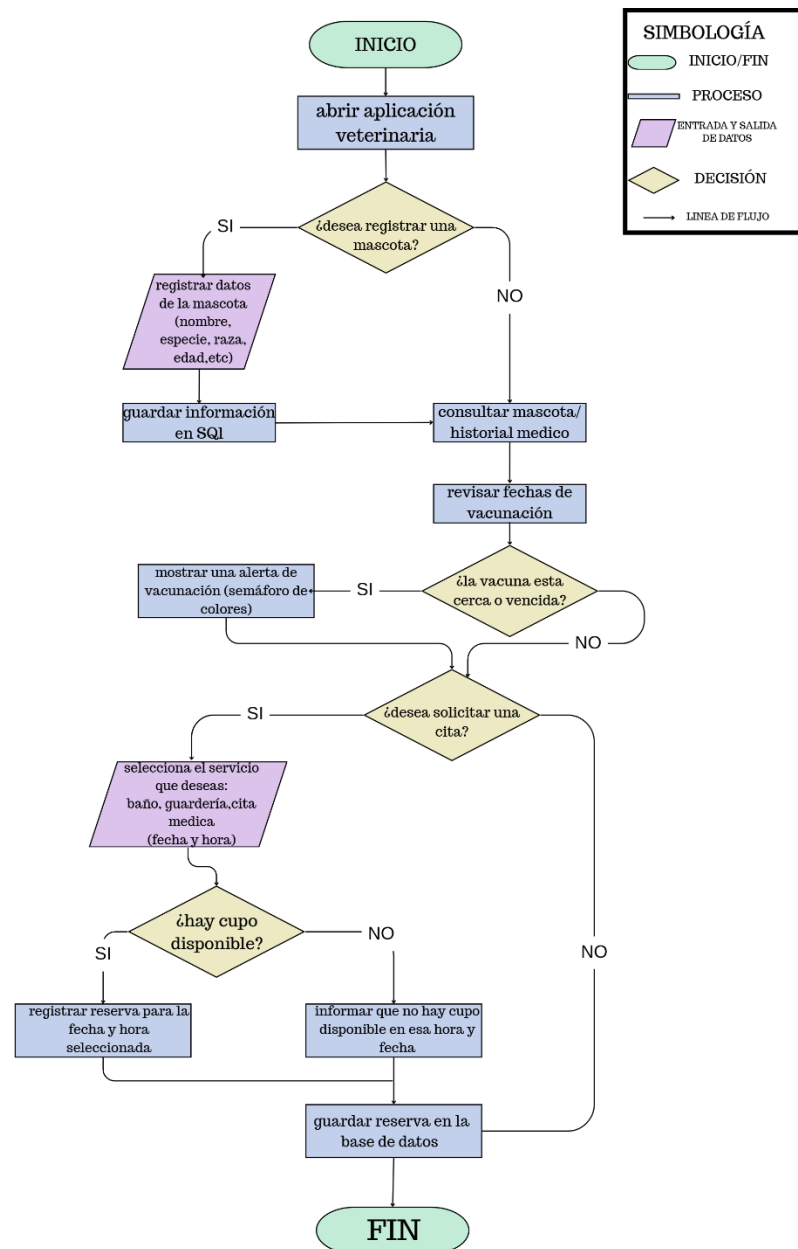


Figura 1. Diagrama de flujo del proceso de navegación, alertas de vacunación y reservas.

Descripción del Proceso: El diagrama de la *Figura 1* detalla la secuencia lógica del sistema. Inicia con el módulo de gestión de perfiles, permitiendo registrar nuevas mascotas con atributos clave (nombre, especie, raza, edad) guardados en SQLite3, o consultar registros previos. Posteriormente, evalúa las fechas de vacunación para emitir una alerta en la GUI mediante un semáforo de colores en caso de vacunas próximas o vencidas. Finalmente, en el módulo de citas (baño, guardería o consulta médica), el sistema valida la disponibilidad de aforo en la fecha y hora seleccionadas. Si hay cupos, confirma la cita y la persiste en la base de datos; de lo contrario, notifica la indisponibilidad de horario.

Avances de Arquitectura

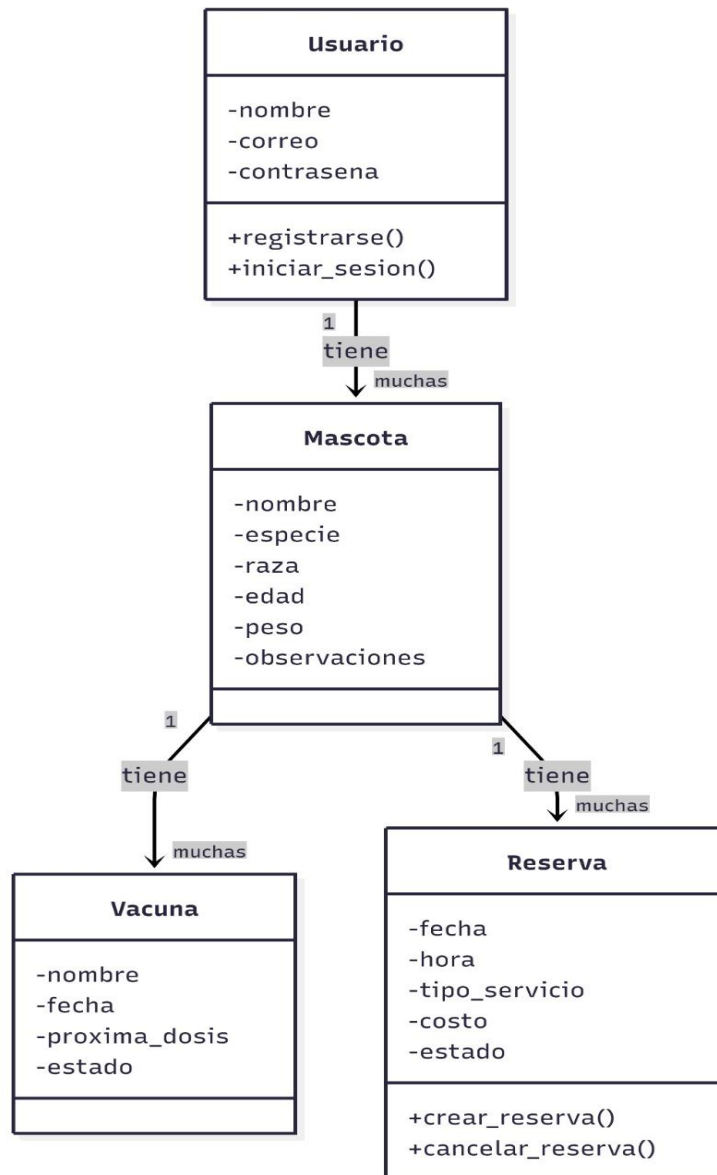


Figura 2. Diagrama de Clases UML del sistema de gestión veterinaria.

Justificación de la Arquitectura: Como se observa en la Figura 2, el dominio del problema se modeló bajo el paradigma de Programación Orientada a Objetos (POO) estructurado en cuatro clases clave:

- **Usuario:** Modela los datos de acceso (nombre, correo, contraseña) y sus operaciones (`registrarse()`, `iniciar_sesion()`). Mantiene una relación de 1 a muchos con la entidad **Mascota**.
- **Mascota:** Entidad central del dominio que almacena atributos biológicos y clínicos (especie, raza, edad, peso, observaciones). Se relaciona de 1 a muchos con las entidades **Vacuna** y **Reserva**.

- **Vacuna:** Administra la trazabilidad del carné de vacunación (nombre, fecha, próxima dosis, estado) para alimentar la lógica del semáforo.
- **Reserva:** Gestiona las agendas del establecimiento (fecha, hora, tipo servicio, costo, estado) y ejecuta los métodos de negocio crear reserva() y cancelar reserva().

Modelo de desarrollo

Se adoptó el Modelo en Cascada debido a la naturaleza académica del proyecto, donde los requerimientos, diagramación y diseño del sistema se fijan de manera secuencial antes de la fase de codificación. Esto garantiza que las entidades, restricciones de base de datos y la interfaz se encuentren completamente especificadas antes de escribir el código fuente en Python.

Historias de usuario:

para la definición de requerimientos funcionales, se definieron las siguientes Historias de Usuario principales:

- Registro e Historial de Mascota
 - Como: Dueño de mascota / Administrador.
 - Quiero: Registrar los datos generales e historial de mi mascota.
 - Para: Mantener centralizada la información médica en el sistema.
- Alerta de Vacunación (Semáforo)
 - Como: Usuario del sistema.
 - Quiero: Visualizar alertas de color según el estado del esquema de vacunas.
 - Para: Evitar olvidar las fechas de refuerzo y mantener al día la salud del animal.
- Agendamiento de Citas y Control de Aforo
 - Como: Cliente de la veterinaria.
 - Quiero: Solicitar citas de baño, guardería o médica verificando cupos disponibles.
 - Para: Agendar servicios sin cruces de horarios ni sobrecupo en el establecimiento.